Investigating the Effect of Context Enrichment and Style Interventions on AI Text Detectability

Yongkyun (Daniel) Lee

Mini Research Project - March 2025

About Me

yongkyun.daniel.lee@gmail.com | linkedin.com/in/yongkyunlee | noninertialframe.com

2016-2018

Sergeant, Republic of Korea Army

SIGINT research

2021

B.S. Computer Science, Caltech

2021-2022

Analyst, Goldman Sachs

Portfolio management tool in Private Wealth Management Team

2022-2026

Software Engineer, Onehouse

Spark Jobs/SQL platform. Quanton Engine optimization. Lakebase.

2026-Present

Founder, Galley

AI team for technical content marketing

tl;dr

Problem

AI-generated technical content is easily detected by both humans and detectors.

Method

Ablation study on the effects of context enrichment, style constraints, and post-hoc humanization on detectability and factual accuracy.

Key Results

All AI conditions showed ~97% GPTZero AI probability vs. 35% for human baselines. Style constraints achieved highest factual precision (99.1%) while humanization degraded accuracy to 97.6%.

Motivation

9x Growth in AI-Generated Content

Explosive growth in low-quality AI content. Ninefold increase in “AI slop” mentions in 2025 vs. 2024

73% of Practitioners Say They Can Identify AI Text

Medical experts correctly identified AI-authored text in ~72% of evaluations in a blinded study.

Quality is the biggest blocker to publishing more technical content

From interviews with 10+ founders and GTM people, low quality of AI-generated blog posts is the issue.

[1] Meltwater (2025). 9x increase in “AI slop” mentions.

[2] Detecting AI-Generated Text (PMC, 2023). Medical experts: ~72% accuracy.

[3] GPTZero Benchmark (2024). 99.3% overall accuracy, 0.24% FPR.

[4] Originality.ai (2024). 99%+ accuracy, <1% FPR.

Literature Review - AI Text Detection

DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature

Mitchell et al. (2023)

Detects AI text zero-shot by measuring log-probability curvature under perturbations. Foundational baseline method used in this study.

A Watermark for Large Language Models

Kirchenbauer et al. (2023)

Partitions vocabulary into green/red token lists during generation, enabling statistical detection. Establishes proactive watermarking approach.

Literature Review - Evasion & Robustness

Can AI-Generated Text be Reliably Detected?
Sadasivan et al. (2023)
Recursive paraphrasing degrades detector accuracy. Proves theoretical impossibility bounds as LLMs improve.

Paraphrasing Evades AI-Generated Text Detectors, but Retrieval is an Effective Defense
Krishna et al. (2023)
11B-param DIPPER paraphraser defeats watermarking, GPTZero, and DetectGPT. Demonstrates fragility of current detection.

HUMPA: Humanized LLM-Paraphraser for Evading AI-Generated Text Detection (ICLR 2025)
Wang et al. (2025)
Trains a proxy LLM via DPO to produce human-like text that evades detectors.

Literature Review - Detector Limitations

GPT Detectors Are Biased Against Non-Native English Writers
Liang et al. (2023)
Detectors show 61% false positive rate on non-native English writing. Highlights equity concerns in detector deployment and evaluation.

DetectRL: Benchmarking LLM-Generated Text Detection in Real-World Scenarios
Wu et al. (2024)
Benchmarks SOTA detectors on real-world text, showing significant performance drops vs. lab conditions. Motivates ecological validity.

Language Models Optimized to Fool Detectors Still Have a Distinct Style (And How to Change It)
Rivera Soto et al. (2025)
Models trained to evade detectors still retain measurable stylistic fingerprints. Supports using linguistic feature analysis as a complementary detection axis.

Literature Review - Accuracy Metrics

Enabling Large Language Models to Generate Text with Citations
Gao et al. (2023)
Proposes citation-level precision and recall metrics for evaluating factual grounding.

Evaluating Verifiability in Generative Search Engines
Liu et al. (2023)
Shows high surface fluency does not guarantee factual accuracy.

Literature Review - Gaps

Existing work addresses detection and evasion separately:

Detection: DetectGPT, watermarking, and trained classifiers achieve high accuracy on unmodified AI text.

Evasion: Paraphrasing (DIPPER, HUMPA) defeats detectors but does not track factual accuracy trade-offs.

Bias: Detectors show systematic biases (e.g., non-native English) and degrade in real-world conditions.

The Gap: No prior work systematically isolates which content generation pipeline decisions reduce detectability while tracking factual accuracy as a co-metric.

This study attempts to fill that gap with a controlled ablation design.

Experimental Design

4-Condition Ablation Study

3 Open Source Projects: CrewAI | DuckDB | LangChain

`context_rich`	**Baseline** - README, source files, GitHub issues, community threads, competitor docs
`style_constrained`	+ Persona prompt, anti-pattern rules, sentence structure rules, content rules
`humanized`	context_rich output rewritten with “sound human” prompt
`human_baseline`	Gold set - real engineer-written blog posts

Style Constraint Design

Persona: “You are a senior backend engineer writing for other engineers on your team’s blog. Sound experienced and opinionated.”

Sentence Structure

Vary sentence length aggressively: mix very short (3-7 words) with longer complex ones (25+ words)

Use contractions naturally: don’t, it’s, we’ve, wouldn’t, can’t, I’m

Start some sentences with I, We, So, But, or And. Use occasional sentence fragments for emphasis.

Anti-Pattern Rules

Never use: Moreover, Furthermore, Additionally, Subsequently, Accordingly, Indeed, Notably

Never open with: “In the world of”, “In today’s”, “Let’s dive in”, “It’s important to note”

Ban overused AI words: delve, leverage, streamline, robust, seamless, cutting-edge, pivotal, holistic

No excessive bullet lists, no em dashes, no hedging (“arguably”, “one might argue”)

Content Rules

Include at least one specific version number, date, or command from provided sources

Only mention errors/configs that appear in provided sources. Do not invent examples.

Include one grounded trade-off judgment and 1-2 mild opinions or subjective judgments

Methodology: Three Evaluation Axes

AI Detectability

GPTZero: gptzero_generated_prob and gptzero_human_prob per document

Each document submitted to GPTZero API (v2/predict/text)

Primary metric: generated_prob (probability text is AI-generated, 0-1)

Factual Accuracy (LLM Judge)

Two-step LLM judge pipeline using Claude Opus

Step 1 - Claim Extraction: Extract all atomic factual claims from generated text

Step 2 - Claim Verification: Judge each claim against reference docs as CORRECT, INCORRECT, or UNVERIFIABLE

Metric: Factual precision = correct_claims / total_claims per document

Linguistic Features

Extract linguistic features per document

Use spaCy en_core_web_sm model

Results: AI Detectability

condition	gptzero_ai_prob_mean	gptzero_ai_prob_std	gptzero_human_prob_mean	gptzero_human_prob_std
context_rich	0.973	0.051	0.000	0.000
style_constrained	0.979	0.046	0.000	0.000
humanized	0.977	0.036	0.000	0.000
human_baseline	0.348	0.381	0.404	0.414

* No significant difference between AI conditions.
* Human-written blog posts are significantly different

Results: Accuracy

condition	model	precision_mean	precision_std	total_claims_mean	incorrect_mean
context_rich	claude-opus-4-6 (Claude Code)	0.971	0.028	49.000	1.250
	codex-cli (Codex CLI)	0.998	0.006	42.833	0.083
style_constrained	claude-opus-4-6 (Claude Code)	0.992	0.015	50.333	0.333
	codex-cli (Codex CLI)	0.990	0.023	44.667	0.417
humanized	claude-opus-4-6 (Claude Code)	0.976	0.021	47.833	1.000

* No significant difference between models or conditions

Results: Linguistic Feature Analysis

condition	model	sent_len_std	sent_len_mean	vocab_diversity	contraction_rate	first_person_rate	discourse_marker_rate	list_density	passive_ratio	paragraph_len_std	specificity_score
context_rich	claude-opus-4-6 (Claude Code)	15.301	25.783	0.727	0.177	0.000	0.003	0.021	0.099	27.764	1.258
	codex-cli (Codex CLI)	13.706	23.263	0.708	0.000	0.000	0.000	0.080	0.100	24.494	1.146
style_constrained	claude-opus-4-6 (Claude Code)	13.075	21.417	0.721	0.955	0.001	0.000	0.005	0.076	28.197	1.410
	codex-cli (Codex CLI)	11.910	19.612	0.709	0.240	0.003	0.000	0.000	0.058	27.107	1.137
humanized	claude-opus-4-6 (Claude Code)	14.443	23.531	0.748	0.959	0.002	0.001	0.000	0.085	26.737	1.152
human_baseline	human	24.467	33.713	0.689	0.351	0.006	0.000	0.174	0.141	19.536	2.228

sent_len_mean: average sentence length. sent_len_std: standard deviation of sentence length. vocab_diversity: unique words / total words

contraction_rate: ratio of contracted (e.g. don't) form. first_person_rate: proportion of first-person words (e.g. we, I)

discourse_marker_rate: occurrences of filler/transition phrases per sentence (e.g., "furthermore", "it's worth noting")

list_density: List items (e.g. bullet points) per paragraph. Passive ratio: fraction of sentences containing passive voice

paragraph_len_std: standard deviation of paragraph lengths (in words). specificity_score: named entities + numeric tokens per sentence

Results & Future Work

Result

Simple prompt engineering is not good enough to avoid AI detection

Prompt for adjusting style does not affect factual accuracy

Linguistic features show some gaps that we can capitalize to avoid detection

Future Work

Try different ways of recursive paraphrasing

Use AI detector's diagnosis in paraphrasing process

Style transfer by providing sample texts as context

Structural approach where the content is built bottom up instead of writing one-shot

Links

Github Link: `github.com/yongkyunlee/ai-detectability-research`

Youtube Link: `youtu.be/hRoQbThAG-U`